# LLORA — Library Assistant Manual

**LLORA** (Lorma Library Online Research Assistant) is an AI-powered chat assistant for the Lorma Colleges library. It integrates with the Koha ILS (Integrated Library System) and gives library patrons a chat widget embedded in the OPAC website. Patrons can search the book catalog, ask about library policies and hours, and request a live librarian — all from the same chat window.

---

## Table of Contents



---

## 1. System Overview

```
Patron Browser
  └── koha-chatbot-inline.js  ← widget injected into Koha OPAC pages
        │
        │  REST API calls
        ▼
  FastAPI Backend (app/main.py)
        │
        ├── Query Classifier   → determines patron intent
        ├── Catalog Handler    → searches Koha OPAC via RSS
        ├── Library Info       → answers FAQ questions
        ├── LLM Client         → OpenRouter (cloud) or Ollama (local)
        ├── Session Store      → SQLite / Turso (persistent)
        └── Ably Client        → real-time push for live chat

Admin Browser
  └── /admin/          ← session monitoring, analytics, settings
  └── /chat/           ← librarian live chat interface
```

**How a patron message is handled:**

1. The widget sends the message to `/api/chat`
2. The query classifier determines intent: `catalog_search`, `library_info`, `greeting`, `conversational`, `unclear`, or `talk_to_librarian`
3. The message is routed to the appropriate handler
4. The reply is stored in the database and returned to the widget

---

## 2. Prerequisites

| Requirement | Notes |
|---|---|
| Python 3.11+ | Required for the backend |
| A Koha ILS instance | Must be reachable from the server |
| OpenRouter API key | Free tier available at openrouter.ai — used for the LLM |
| Turso account | Free tier at turso.tech — required for Vercel deployment |
| Ably account | Free tier at ably.com — optional, for real-time live chat |
| Vercel account | For cloud deployment |

For local development only, you can skip Turso and Ably — SQLite and HTTP polling work out of the box.

---

## 3. Environment Variables

Copy `.env.example` to `.env` and fill in the values.

### Required

| Variable | Description | Example |
|---|---|---|
| `KOHA_API_URL` | Base URL of your Koha OPAC | `http://your-koha-server` |
| `ADMIN_API_KEY` | Secret key for admin dashboard access | `change-me-to-something-secret` |

### LLM (choose one)

| Variable | Description | Example |
|---|---|---|
| `OPENROUTER_API_KEY` | OpenRouter API key (recommended for cloud) | `sk-or-v1-...` |
| `OLLAMA_URL` | Ollama endpoint (for local LLM) | `http://localhost:11434/v1` |
| `OLLAMA_MODEL` | Model name | `llama3.2:3b` |

When `OPENROUTER_API_KEY` is set, the system uses OpenRouter automatically. The primary model is `meta-llama/llama-3.2-3b-instruct:free` with 4 free fallback models.

### Admin Login

| Variable | Description | Default |
|---|---|---|
| `ADMIN_USERNAME` | Admin dashboard login username | `admin` |
| `ADMIN_PASSWORD` | Admin dashboard login password | `admin` |

**Change these from the defaults before going to production.**

### Database

| Variable | Description | Default |
|---|---|---|
| `SESSION_DB_PATH` | Path to local SQLite file | `/tmp/sessions.db` |
| `TURSO_DATABASE_URL` | Turso database URL (for Vercel) | *(optional)* |
| `TURSO_AUTH_TOKEN` | Turso auth token | *(optional)* |

When both Turso variables are set, the app uses Turso instead of local SQLite. This is required on Vercel since the filesystem is ephemeral.

### Email Notifications

Two options for sending librarian handoff emails:

**Option A — Google Workspace service account (recommended):**

| Variable | Description |
|---|---|
| `GOOGLE_SERVICE_ACCOUNT_JSON` | Full service account JSON as a string |
| `GOOGLE_SERVICE_ACCOUNT_FILE` | Path to service account JSON file (alternative) |
| `SMTP_EMAIL` | The Workspace email the service account delegates to |

**Option B — Gmail SMTP with App Password:**

| Variable | Description |
|---|---|
| `SMTP_EMAIL` | Your Gmail address |
| `SMTP_PASSWORD` | Gmail App Password (not your regular password) |

**Fallback:**

| Variable | Description |
|---|---|
| `LIBRARIAN_EMAIL` | Fallback email if no staff contacts are configured |

### Other

| Variable | Description | Default |
|---|---|---|
| `CHATBOT_PUBLIC_URL` | Public URL of the chatbot (used in email links) | `http://localhost:8000` |
| `ABLY_API_KEY` | Ably API key for real-time messaging | *(optional)* |
| `NTFY_TOPIC` | ntfy.sh topic for push notifications | *(optional)* |
| `MESSENGER_LINK` | Facebook Messenger link for the library page | *(optional)* |

---

## 4. Local Development Setup

```
# 1. Clone the repository
git clone <repo-url>
cd koha_chatbot

# 2. Create and activate a virtual environment
python -m venv .venv
source .venv/bin/activate        # Linux / macOS
# .venv\Scripts\activate         # Windows

# 3. Install dependencies
pip install -r requirements.txt

# 4. Configure environment
cp .env.example .env
# Edit .env — at minimum set KOHA_API_URL and ADMIN_API_KEY

# 5. Start the server
uvicorn app.main:app --reload
```

The app is now running at `http://localhost:8000`.

- Chat widget: `http://localhost:8000/static/index.html`
- Admin dashboard: `http://localhost:8000/admin/`
- Health check: `http://localhost:8000/health`

> **Note:** Without `OPENROUTER_API_KEY` or a local Ollama instance, the LLM features (conversational replies, catalog result formatting) will be disabled. The bot will still answer FAQ questions and search the catalog using keyword matching.

## 5. Deploying to Vercel

### 5.1 Set up Turso

1. Sign up at turso.tech
2. Create a new database: `turso db create koha-chatbot`
3. Get the URL: `turso db show koha-chatbot --url`
4. Get a token: `turso db tokens create koha-chatbot`

### 5.2 Deploy

```
# Install Vercel CLI
npm i -g vercel

# Deploy
vercel --prod
```

### 5.3 Set environment variables in Vercel

Go to your Vercel project → Settings → Environment Variables and add:

| Variable | Value |
|---|---|
| `KOHA_API_URL` | Your Koha server URL |
| `ADMIN_API_KEY` | A strong secret key |
| `ADMIN_USERNAME` | Your admin username |
| `ADMIN_PASSWORD` | Your admin password |
| `OPENROUTER_API_KEY` | Your OpenRouter key |
| `TURSO_DATABASE_URL` | `libsql://your-db.turso.io` |
| `TURSO_AUTH_TOKEN` | Your Turso token |
| `CHATBOT_PUBLIC_URL` | `https://your-project.vercel.app` |
| `SMTP_EMAIL` | *(optional)* For email notifications |
| `SMTP_PASSWORD` | *(optional)* Gmail App Password |
| `ABLY_API_KEY` | *(optional)* For real-time live chat |

### 5.4 Verify

Visit `https://your-project.vercel.app/health` — you should see:

```
{"status": "ok", "llm_enabled": true, ...}
```

---

## 6. Self-Hosted Deployment (Apache + Koha)

This setup runs the chatbot on the same server as Koha and proxies it through Apache so it shares the same origin as the OPAC (no CORS issues).

### 6.1 Run the backend

```
uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Use a process manager like `systemd` or `supervisor` to keep it running.

### 6.2 Configure Apache

Add the contents of `apache/chatbot-proxy.conf` inside your Koha OPAC `<VirtualHost>` block:

```
ProxyPreserveHost On
ProxyPass /chatbot/ http://127.0.0.1:8000/
ProxyPassReverse /chatbot/ http://127.0.0.1:8000/
```

Make sure `mod_proxy` and `mod_proxy_http` are enabled:

```
a2enmod proxy proxy_http
systemctl reload apache2
```

The chatbot is now accessible at `http://your-koha-server/chatbot/`.

### 6.3 Update the widget URL

In `app/static/koha-embed.js`, the `CHATBOT_URL` is already set to `/chatbot` by default — no change needed if you used the proxy path above.

---

## 7. Embedding the Widget in Koha

### Option A: Inline widget (recommended)

This injects the chat widget directly into every Koha OPAC page. No iframe, no cross-origin issues.

In Koha: **Administration → System Preferences → OPAC → OPACUserJS**

Paste this snippet and update the URL to point to your chatbot:

```
(function () {
  var s = document.createElement("script");
  s.src = "https://your-chatbot-url/static/koha-chatbot-inline.js";
  s.async = true;
  document.body.appendChild(s);
})();
```

For self-hosted with the Apache proxy:

```
(function () {
  var s = document.createElement("script");
  s.src = "/chatbot/static/koha-chatbot-inline.js";
  s.async = true;
  document.body.appendChild(s);
})();
```

### Option B: iframe embed

Adds the widget inside an iframe. Simpler but slightly more limited.

```
(function () {
  var s = document.createElement("script");
  s.src = "https://your-chatbot-url/static/koha-embed.js";
  s.async = true;
  document.body.appendChild(s);
})();
```

### What the widget does

- Shows a floating button (bottom-right) with the LLORA avatar
- On first open, asks the patron to identify themselves (Student, Faculty, Alumni, Visitor)
- Loads FAQ quick-reply buttons from `/api/faqs`
- Checks librarian availability and enables/disables the Librarian button accordingly
- Persists the session across page navigations using `sessionStorage`

## 8. Admin Dashboard

Access the admin dashboard at `/admin/` (e.g. `https://your-chatbot-url/admin/`).

Log in with your `ADMIN_USERNAME` and `ADMIN_PASSWORD`.

### Dashboard sections

| Section | Description |
|---|---|
| **Sessions** | Browse all patron chat sessions, search by keyword, view full transcripts |
| **Analytics** | Intent breakdown, hourly/daily activity charts, avg messages per session |
| **Quality** | Patron feedback (thumbs up/down), unanswered query review |
| **Live Chat** | Active librarian handoff queue, live chat history |
| **Staff Ratings** | Per-librarian satisfaction scores from patron ratings |
| **AI Settings** | Configure bot name, personality, limitations, welcome message |
| **Library Info** | Manage FAQ buttons shown in the widget |
| **Library Hours** | Set opening hours per day (controls librarian button availability) |
| **Staff Contacts** | Add librarian emails for handoff notifications |
| **Settings** | Notification emails, feature toggles |

### Session management

- Click any session to view the full conversation transcript
- Flag sessions with a note for follow-up
- Export sessions to CSV (filtered by status or date range)
- Bulk delete expired sessions older than N days

---

## 9. AI Settings

Go to **Admin → AI Settings** to configure the bot's personality without touching code.

| Field | Description | Default |
|---|---|---|
| **Assistant Name** | The name shown in the widget header and used in responses | `LLORA` |
| **Personality** | How the bot speaks — tone, style, emoji usage | Warm and concise |
| **Limitations** | Topics the bot is restricted to | Library and school topics only |
| **Welcome Message** | Shown when a patron opens the chat. Use `{name}` to insert the bot name | Standard LLORA greeting |

Changes take effect immediately — no restart needed.

---

## 10. Library Info & FAQ Management

Go to **Admin** → **Library Info** to manage the FAQ buttons shown in the chat widget.

Each FAQ item has:

| Field | Description |
|---|---|
| **Label** | Button text shown in the widget (e.g. `🕐 Library hours`) |
| **Question** | The question text sent when the patron clicks the button |
| **Answer** | The bot's reply text |
| **Image** | Optional image shown with the reply (max 200 KB, compressed client-side) |
| **PDF** | Optional PDF shown as a download button (max 5 MB) |

### How FAQ matching works

When a patron types a question (not just clicking a button), the system:

1. Checks for an **exact match** against all FAQ questions — returns the answer directly, no LLM needed
2. If no exact match, scores all FAQs by **keyword overlap** and picks the top 3
3. Uses the LLM to generate a **conversational reply** from the matched FAQ content
4. If no match at all, replies: *"I'm sorry, I don't have that information available. Please contact library staff."*

---

## 11. Library Hours Configuration

Go to **Admin** → **Library Hours** to set opening hours for each day of the week.

Each day can have one or more time windows in `HH:MM` format (24-hour). An empty list means the library is closed that day.

**Example:** - Monday–Friday: `07:00` – `19:00` - Saturday: `08:30` – `16:30` - Sunday: *(closed)*

### How hours affect the widget

- The **Librarian button** is automatically disabled 30 minutes before closing time
- When the library is closed, the widget shows an after-hours message with the next available time
- The system uses **Asia/Manila timezone (UTC+8)** — adjust your hours accordingly if your library is in a different timezone

> To change the timezone, edit `_check_librarian_available()` in `app/main.py` and update the `ph_tz` offset.

---

## 12. Staff Contacts & Notifications

Go to **Admin** → **Staff Contacts** to add librarian names and emails.

When a patron requests a librarian:

1. All active staff contacts receive a **personalized email** with a link to the live chat queue
2. The first librarian to click **Claim** gets the session
3. All other librarians receive a **"already claimed"** email so they know not to join
4. If no staff contacts are configured, the fallback `LIBRARIAN_EMAIL` env variable is used

### ntfy.sh push notifications (optional)

Set `NTFY_TOPIC` in your `.env` to also send a push notification via ntfy.sh when a patron requests a librarian. This works on Android and iOS with the ntfy app.

---

## 13. Live Chat (Librarian Handoff)

### Patron side

1. Patron clicks the **Librarian** button in the chat widget
2. A patron identity form appears (patron type + details)
3. After submitting, the bot sends: *"I've notified a librarian — they'll join this chat shortly!"*
4. The patron can cancel the request before a librarian joins
5. Once a librarian joins, messages go directly to the librarian (the AI is bypassed)
6. After the librarian ends the chat, the bot returns: *"The librarian has ended the chat. I'm LLORA, your AI assistant — what else can I help you with?"*
7. The patron is shown a **satisfaction rating** (1–4 scale)

### Librarian side

1. Open the live chat page at `/chat/` (or `/admin/chat/`)
2. Enter your name when prompted
3. The queue shows all waiting patrons with their display name and message count
4. Click **Claim** to take a session — other librarians are notified automatically
5. Type replies in the message box — the patron sees them in real time
6. Click **End Chat** when done

### Live chat message flow

```
Patron types → /api/chat → saved to live_chat_messages table
                         → published to Ably channel (instant)
                         → available via /api/poll (fallback)

Librarian types → /admin/api/live-chat/{id}/reply
                → saved to live_chat_messages table
                → published to Ably channel (instant)
                → available via /admin/api/live-chat/{id}/messages (fallback)
```

---

## 14. Real-Time Messaging with Ably

Without Ably, the widget polls for new messages every 2 seconds. With Ably, messages are delivered instantly via WebSocket.

### Setup

1. Sign up at ably.com — the free tier supports up to 6 million messages/month
2. Create an app and copy the API key
3. Set `ABLY_API_KEY` in your environment variables

### How it works

- The backend publishes to Ably channels via the REST API after saving each message to the DB
- The widget subscribes to `live-chat:{live_chat_id}` for patron ↔ librarian messages
- The librarian dashboard subscribes to `handoff-queue` for new patron waiting notifications
- If Ably is not configured or a publish fails, polling continues as the fallback — no messages are lost

---

## 15. Database

### Local SQLite (development / self-hosted)

The database is created automatically at the path set by `SESSION_DB_PATH` (default: `/tmp/sessions.db`).

For self-hosted production, set a persistent path:

```
SESSION_DB_PATH=/var/lib/koha-chatbot/sessions.db
```

### Turso (Vercel / cloud)

Turso is a cloud SQLite service with an HTTP API — no native binaries needed, which makes it compatible with Vercel's serverless environment.

The app uses the same SQL queries for both backends. The `db.py` module transparently routes to Turso when `TURSO_DATABASE_URL` and `TURSO_AUTH_TOKEN` are set.

### Schema

The database has these tables:

| Table | Description |
| --- | --- |
| `sessions` | One row per patron session — metadata, handoff state, display name |
| `messages` | All chat messages with role, content, timestamp, and intent |
| `feedback` | Patron thumbs up/down ratings on bot responses |
| `session_flags` | Admin notes flagged on sessions |
| `staff_ratings` | Patron satisfaction ratings for librarian interactions (1–4 scale) |
| `live_chat_sessions` | Separate sessions for librarian handoffs |
| `live_chat_messages` | Messages exchanged during live chat |
| `dashboard_settings` | Key-value store for AI settings, library info, hours, staff contacts |
| `schema_meta` | Schema version tracking for fast migration checks |

### Migrations

Schema migrations run automatically on startup. The current schema version is `7`. If the DB is already at version 7, migrations are skipped entirely (single cheap query check).

---

## 16. Project Structure

```
koha_chatbot/
├── api/
│   └── index.py                  # Vercel entry point — imports FastAPI app
├── app/
│   ├── main.py                   # FastAPI app, /api/chat, startup lifecycle
│   ├── config.py                 # Environment variable loading
│   ├── models.py                 # Pydantic data models
│   ├── db.py                     # SQLite / Turso database abstraction
│   ├── groq_client.py            # LLM client (OpenRouter / Ollama)
│   ├── query_classifier.py       # Intent classification
│   ├── catalog_handler.py        # Koha catalog search via RSS
│   ├── library_info_handler.py   # FAQ matching and response generation
│   ├── session_manager.py        # In-memory conversation history
│   ├── session_store.py          # Persistent session storage (SQLite/Turso)
│   ├── staff_store.py            # Dashboard settings storage
│   ├── admin_auth.py             # Admin API key authentication
│   ├── admin_routes.py           # Admin API endpoints
│   ├── staff_routes.py           # Staff settings API endpoints
│   ├── ai_settings.py            # AI personality configuration
│   ├── ably_client.py            # Ably real-time publish helper
│   ├── email_notify.py           # Librarian email notifications
│   └── static/
│       ├── index.html            # Standalone chat widget page
│       ├── admin.html            # Admin monitoring dashboard
│       ├── live-chat.html        # Librarian live chat interface
│       ├── koha-chatbot-inline.js # Inline widget (recommended embed)
│       └── koha-embed.js         # iframe-based embed (alternative)
├── apache/
│   └── chatbot-proxy.conf        # Apache reverse proxy config for Koha
├── tests/
│   ├── test_api_endpoint.py      # API endpoint tests
│   └── test_catalog_handler.py   # Catalog handler tests (property-based)
├── vercel.json                   # Vercel deployment config
├── requirements.txt              # Python dependencies
├── .env.example                  # Environment variable template
└── MANUAL.md                     # This file
```

## 17. Running Tests

```
# Run all tests
pytest

# Run with verbose output
pytest -v

# Run a specific test file
pytest tests/test_api_endpoint.py
```

The test suite uses Hypothesis for property-based testing — it generates hundreds of random inputs to verify that the catalog handler and API always return valid responses.

---

## 18. Troubleshooting

### Widget shows old version after deployment

The widget JS is served with `Cache-Control: no-cache` headers. If you still see an old version:

1. Hard refresh the browser (`Ctrl+Shift+R` / `Cmd+Shift+R`)
2. Check that Vercel has finished deploying (check the Vercel dashboard)
3. If the Koha site has its own caching layer (Varnish, CDN), purge the cache for `/static/koha-chatbot-inline.js`

### LLM not responding

Check `/health` — if `llm_enabled` is `false`, the `OPENROUTER_API_KEY` is not set or not being read.

On Vercel, verify the environment variable is set in the project settings (not just in `.env`).

### Catalog search returns no results

1. Visit `/debug/koha-test` to test the Koha connection directly
2. Check that `KOHA_API_URL` points to the correct Koha server
3. Some Koha servers have a WAF that blocks automated requests — the widget will automatically fall back to client-side search in this case

### Librarian emails not sending

1. Check that `SMTP_EMAIL` is set

2. For Gmail SMTP: make sure you're using an **App Password**, not your regular Gmail password. Generate one at Google Account → Security → 2-Step Verification → App Passwords
3. For service account: verify the service account has domain-wide delegation enabled and the `https://www.googleapis.com/auth/gmail.send` scope is authorized

### Live chat messages not appearing in real time

If Ably is not configured, the widget falls back to polling every 2 seconds — messages will still arrive, just with a short delay. To enable real-time delivery, set `ABLY_API_KEY`.

### Database errors on Vercel

Vercel's filesystem is ephemeral — local SQLite files are lost between requests. Make sure `TURSO_DATABASE_URL` and `TURSO_AUTH_TOKEN` are set in your Vercel environment variables.

### Admin dashboard login fails

- Verify `ADMIN_USERNAME` and `ADMIN_PASSWORD` match what you set in the environment variables
- Verify `ADMIN_API_KEY` is set — the login endpoint returns it after successful authentication
- On Vercel, check that all three variables are set in the project settings

### Sessions show as expired immediately

The session timeout is 5 minutes of inactivity (defined in `session_manager.py`). Active live chat sessions are kept alive automatically. If regular sessions are expiring too fast, this is expected behavior — the admin dashboard shows both active and expired sessions.

---

*Manual version: May 2026 — matches schema version 7*